



ClickHouse для инженеров и архитекторов БД

Словари, оконные и табличные функции



Проверить, идет ли запись

Меня хорошо видно && слышно?



Ставим "+", если все хорошо
"-", если есть проблемы

Тема вебинара

Словари, оконные и табличные функции



Мунир Рафиков

Архитектор

- 5 лет работы архитектором (хранилища данных и enterprise решения)
- 5 лет presale архитектором и системным аналитиком (хранилища данных и enterprise решения)
- 3 года дата-инженером
- Много лет в эксплуатации highload систем

 @tearchit

 moonir@mail.ru

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Карта курса



Темы модуля «Работа с ClickHouse»

Язык запросов SQL

Функции для работы с типами данных, агрегатные функции и UDF

Движки MergeTree Family

Другие движки

Словари, оконные и табличные функции

Джоины и агрегации

Сессия Q&A

Маршрут вебинара



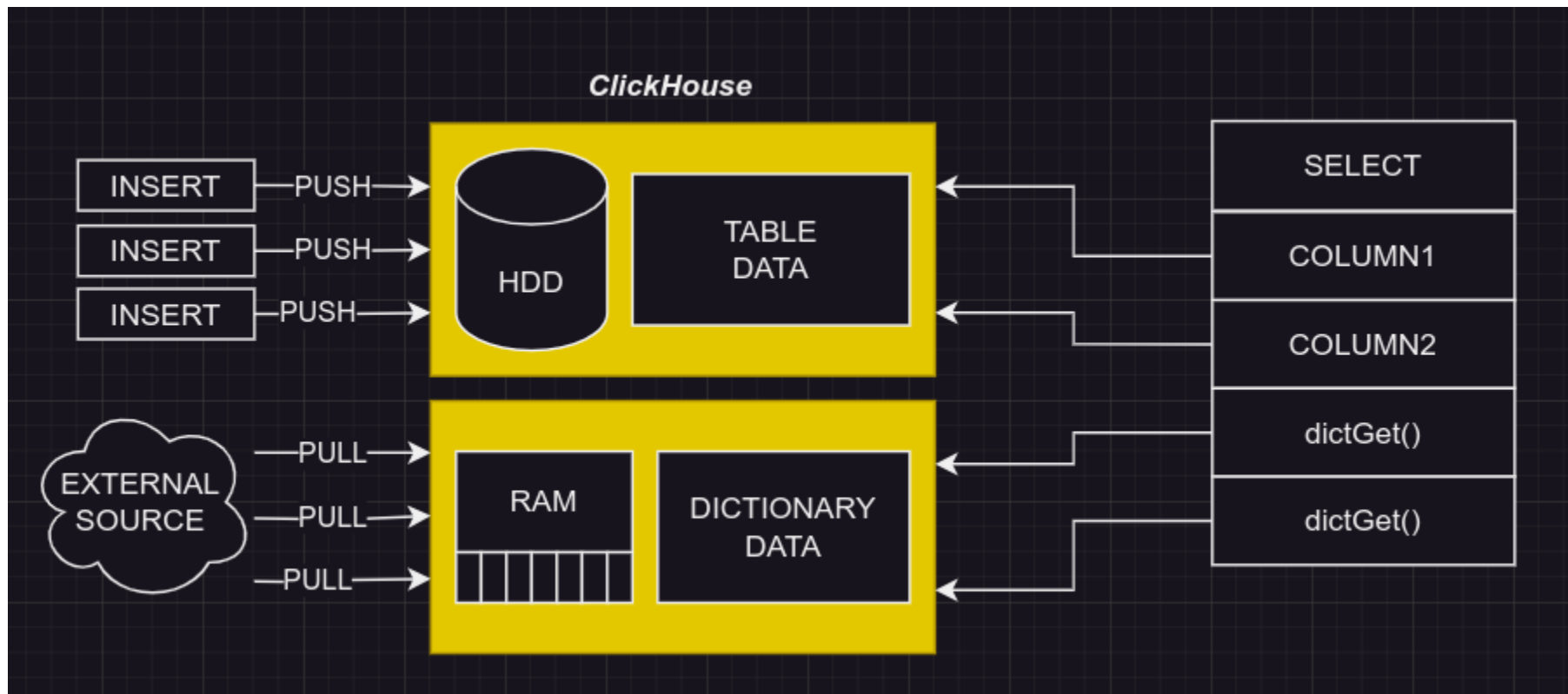
Цели вебинара

К концу занятия вы сможете

1. создавать словари из разных источников
 2. познакомиться с оконными функциями
-

Словари

Что такое словарь



Словарь - данные из внешних систем, которые регулярно забираются ClickHouse-ом в оперативную память (как правило в оперативную, но не только), для использования совместно с собственными данными в запросах [dictionaries_lazy_load](#) - варианты загрузки при старте сервера. По-умолчанию True-при первом использовании.

Структура словаря

KEY	ATTR1	ATTR2	ATTR3
key1	value	value	value
key2	value	value	value
key3	value	value	value
key4	value	value	value

Словарь представляет собой KEY-value хранилище, ключ может быть простым или составным.

Ключ при обращении к словарю указывается целиком.

Атрибутов может быть от 1 и больше, их значения можно получать по ключу.

Зачем нужны словари?

1) Мутабельные данные

В ClickHouse сохраняются события, например действия пользователя, за долгий период. События не подлежат изменению, они уже произошли, поэтому они отлично подходят для хранения и анализа в ClickHouse.

Однако, у пользователя есть так же меняющиеся свойства, такие как статус активности, баланс, семейное положение, и другие. Такие свойства в силу изменчивости не подходят к хранению в ClickHouse, однако так же относятся к этому пользователю.

Для совместного использования тех и других данных и применяются словари.

Зачем нужны словари?

2) Интеграции с другими системами

Описанный в п.1 сценарий, подразумевает хранение мутабельных данных в других системах, и как правило там они изначально и находятся, словари подходят для интеграции этих данных с данными ClickHouse.

Зачем нужны словари?

3) Производительная альтернатива JOIN

Даже если данные уже в ClickHouse, однако в разных таблицах, построение словаря вокруг меньшей таблицы или вокруг её агрегата является более производительным решением.

В реляционных системах мы привыкли JOIN-ить сотни и тысячи строк, иногда десятки тысяч, не принимая в расчет накладные расходы на построение hash-таблицы JOIN-а, и размер оперативной памяти и издержки по CPU необходимые для такой операции. Работая в ClickHouse с миллионами и миллиардами строк, очень быстро становится очевидно дороговизна такого подхода.

Как создать словарь

XML-конфигурация

```
<dictionary>  
<name> ИМЯ СЛОВАРЯ </name>  
<structure>  
КЛЮЧ И АТРИБУТЫ  
</structure>  
<source>  
ИСТОЧНИК  
</source>  
<layout>  
СХЕМА ХРАНЕНИЯ  
</layout>  
<lifetime>  
КАК ЧАСТО ОБНОВЛЯТЬ  
</lifetime>  
</dictionary>
```

SQL-синтаксис

```
CREATE DICTIONARY ИМЯ СЛОВАРЯ  
(  
    АТРИБУТЫ  
)  
PRIMARY KEY КЛЮЧ  
SOURCE(ИСТОЧНИК)  
LAYOUT(СХЕМА ХРАНЕНИЯ)  
LIFETIME(КАК ЧАСТО ОБНОВЛЯТЬ)
```

Как создать словарь / структура словаря

```
--<id><name>Id</name></id> -- если ключ UInt64
<key>
<attribute>
<name>key1</name><type>String</type>
</attribute>
<attribute>
<name>key2</name><type>UInt32</type>
</attribute>
</key>
<attribute>
<name>field1</name><type>String</type>
</attribute>
<attribute>
<name>field2</name><type>String</type>
</attribute>
```

Для SQL простой и составной ключи объявляются одинаково в списке полей и дополнительно перечисляются в PRIMARY KEY:

```
CREATE DICTIONARY имя (
    key1 String,
    key2 UInt32
    field1 String,
    field2 String
)
PRIMARY KEY key1, key2
---- PRIMARY KEY id -- если ключ UInt64
```


Как создать словарь

XML-конфигурация

```
<dictionary>  
<name> ИМЯ СЛОВАРЯ </name>  
<structure>  
КЛЮЧ И АТТРИБУТЫ  
</structure>  
<source>  
ИСТОЧНИК  
</source>  
<layout>  
СХЕМА ХРАНЕНИЯ  
</layout>  
<lifetime>  
КАК ЧАСТО ОБНОВЛЯТЬ  
</lifetime>  
</dictionary>
```

SQL-синтаксис

```
CREATE DICTIONARY ИМЯ СЛОВАРЯ  
(  
    АТТРИБУТЫ  
)  
PRIMARY KEY КЛЮЧ  
SOURCE(ИСТОЧНИК)  
LAYOUT(СХЕМА ХРАНЕНИЯ)  
LIFETIME(КАК ЧАСТО ОБНОВЛЯТЬ)
```

Как создать словарь / источник

В качестве источников поддерживается:

1) файл на файловой системе

для XML:

<source>

<file>

<path>/opt/dictionaries/os.tsv</path>

<format>TabSeparated</format>

</file>

</source>

для SQL:

SOURCE(FILE(path './user_files/os.tsv' format
'TabSeparated'))

Как создать словарь / источник

2) файл доступный по http/https

XML:

```
<source>
  <http>
    <url>http://url/os.tsv</url>
    <format>TabSeparated</format>
    <credentials> -- необязательно
      <user>...</user>
      <password>...</password>
    </credentials>
    <headers> --необязательно
      <header>
        <name>...</name><value>...</value>
      </header>
    </headers>
  </http>
</source>
```

SQL:

```
SOURCE(HTTP(
  url 'http://[::1]/os.tsv'
  format 'TabSeparated'
  credentials(user 'user' password
'password')
  headers(header(name 'API-KEY' value
'key'))
))
```



Как создать словарь / источник

3) другие базы данных

здесь конфигурация через SQL и XML
однотипна

3.1) *odbc* коннектор

```
SOURCE(ODBC(  
db 'DatabaseName'  
table 'SchemaName.TableName'  
connection_string  
'DSN=some_parameters'  
invalidate_query '...'  
query 'SELECT id, value_1, value_2  
FROM db_name.table_name'  
))
```

db — имя базы данных. Не указывать, если имя базы задано в параметрах.

<connection_string>.

table — имя таблицы и схемы, если она есть.

connection_string — строка соединения.

invalidate_query — запрос для проверки статуса словаря. Необязательный параметр.

query — пользовательский запрос. Необязательный параметр.

Как создать словарь / источник

3.2) *mysql*

```
SOURCE(MYSQL(  
port 3306  
user 'clickhouse'  
password 'qwerty'  
replica(host 'example01-1' priority 1)  
replica(host 'example01-2' priority 1)  
db 'db_name'  
table 'table_name'  
where 'id=10'  
invalidate_query 'Select max(change_date) from...'  
fail_on_connection_loss 'true'  
query 'SELECT id, value_1, value_2 FROM db_name.table_name'))
```



Как создать словарь / источник

3.2) *mysql*

port — порт сервера MySQL. Можно указать для всех реплик или для каждой в отдельности (внутри <replica>).

user — имя пользователя MySQL. Можно указать для всех реплик или для каждой в отдельности (внутри <replica>).

password — пароль пользователя MySQL. Можно указать для всех реплик или для каждой в отдельности (внутри <replica>).

replica — блок конфигурации реплики. Блоков может быть несколько.

db — имя базы данных.

table — имя таблицы.

where — условие выбора.

invalidate_query — запрос для проверки статуса словаря. Необязательный параметр.

fail_on_connection_loss — параметр конфигурации, контролирующий поведение сервера при потере соединения. Если значение true, то исключение генерируется сразу. Если значение false, то сервер попытается выполнить запрос три раза прежде чем сгенерировать исключение. Повторные попытки увеличивают время выполнения запроса. Значение по умолчанию: false.

query — пользовательский запрос. Необязательный параметр.

Как создать словарь / источник

3.3) postgresql

```
SOURCE(POSTGRESQL(  
port 5432  
host 'postgresql-hostname'  
user 'postgres_user'  
password 'postgres_password'  
db 'db_name'  
table 'table_name'  
replica(host 'example01-1' port 5432 priority 1)  
replica(host 'example01-2' port 5432 priority 2)  
where 'id=10'  
invalidate_query 'select max(date_from)...'  
query 'SELECT id, value_1, value_2 FROM db_name.table_name'  
))
```

параметры как у mysql



Как создать словарь / источник

3.4) ClickHouse

```
SOURCE(CLICKHOUSE(  
  host 'example01-01-1'  
  port 9000  
  user 'default'  
  password ''  
  db 'default'  
  table 'ids'  
  where 'id=10'  
  secure 1  
  query 'SELECT id, value_1, value_2 FROM default.ids'  
))
```

аналогично mysql и postgresql, можно включить ssl флагом secure, нельзя указать несколько реплик, предлагается вместо этого использовать Distributed-таблицу

Как создать словарь / источник

3.5) MongoDB

SOURCE(MONGODB(

host 'localhost'

port 27017

user ''

password ''

db 'test'

collection 'dictionary_source'

))

ещё меньше параметров подключения, есть параметр collection - аналог таблицы в MongoDB

Как создать словарь / источник

3.6) Redis

```
SOURCE(REDIS(  
  host 'localhost'  
  port 6379  
  storage_type 'simple'  
  db_index 0  
))
```

host – хост Redis.

port – порт сервера Redis.

storage_type – способ хранения ключей. Необходимо использовать **simple** для источников с одним столбцом ключей, **hash_map** – для источников с двумя столбцами ключей. Источники с более, чем двумя столбцами ключей, не поддерживаются. Значение по умолчанию **simple**.

db_index – номер базы данных. Значение по умолчанию 0.

Как создать словарь / источник

3.7) Cassandra

SOURCE(CASSANDRA(

параметры
описываются
аналогичным
образом))

host – имя хоста с установленной Cassandra или разделенный через запятую список хостов.

port – порт на серверах Cassandra. Если не указан, используется значение по умолчанию: 9042.

user – имя пользователя для соединения с Cassandra.

password – пароль для соединения с Cassandra.

keyspace – имя keyspace (база данных).

column_family – имя семейства столбцов (таблица).

allow_filtering – флаг, разрешающий или не разрешающий потенциально дорогостоящие условия на кластеризации ключевых столбцов. Значение по умолчанию: 1.

partition_key_prefix – количество партиций ключевых столбцов в первичном ключе таблицы Cassandra. Необходимо для составления ключей словаря. Порядок ключевых столбцов в определении словаря должен быть таким же, как в Cassandra. Значение по умолчанию: 1 (первый ключевой столбец - это ключ партицирования, остальные ключевые столбцы - ключи кластеризации).

consistency – уровень консистентности. Возможные значения: One, Two, Three, All, EachQuorum, Quorum, LocalQuorum, LocalOne, Serial, LocalSerial. Значение по умолчанию: One.

where – опциональный критерий выборки.

max_threads – максимальное количество тредов для загрузки данных из нескольких партиций в словарь.

query – пользовательский запрос. Необязательный параметр.

Как создать словарь

XML-конфигурация

```
<dictionary>  
<name> ИМЯ СЛОВАРЯ </name>  
<structure>  
КЛЮЧ И АТРИБУТЫ  
</structure>  
<source>  
ИСТОЧНИК  
</source>  
<layout>  
<схема> параметры </схема>  
</layout>  
<lifetime>  
КАК ЧАСТО ОБНОВЛЯТЬ  
</lifetime>  
</dictionary>
```

SQL-синтаксис

```
CREATE DICTIONARY ИМЯ СЛОВАРЯ  
(  
    АТРИБУТЫ  
)  
PRIMARY KEY КЛЮЧ  
SOURCE(ИСТОЧНИК)  
LAYOUT(схема(параметр1 значение  
параметр2 значение ...))  
LIFETIME(КАК ЧАСТО ОБНОВЛЯТЬ)
```



Схема хранения

flat

предназначена для хранения словарей с простым UInt64 ключом

параметры:

- * `initial_array_size` - по умолчанию 1024, под сколько ключей резервировать память сразу

- * `max_array_size` - по умолчанию 5000000, максимальное количество ключей, при превышении словарь не создается

самая простая и производительная схема, не эффективна по памяти

hashed

строит хеш-таблицу вокруг хеша ключа, параметров нет, несколько эффективнее по памяти

sparse_hashed

тоже самое, но ещё меньше потребление памяти ценой большей нагрузки на CPU

complex_key_hashed, complex_key_sparse_hashed - нет параметров, строит хеш-таблицу вокруг составного или не-UInt64 ключа

Схема хранения

cache

Словарь хранится в кэше, состоящем из фиксированного количества ячеек. Ячейки содержат часто используемые элементы.

При поиске в словаре сначала просматривается кэш. На каждый блок данных, все не найденные в кэше или устаревшие ключи запрашиваются у источника с помощью `SELECT attrs... FROM db.table WHERE id IN (k1, k2, ...)`. Затем, полученные данные записываются в кэш.

Наименее производительный словарь, однако позволяет держать не полный набор данных, а только часто используемые.

параметры

- * `size_in_cells` - размер кеша
- * `allow_read_expired_keys` - возвращать устаревшие
- * `max_update_queue_size` - очередь на обновление
- * `update_queue_push_timeout_milliseconds` - таймаут для очереди
- * `query_wait_timeout_milliseconds` - таймаут для обновления
- * `max_threads_for_updates` - тредов для обновления

Схема хранения

complex_key_cache

аналогично **cache**, для составных ключей

ssd_cache, complex_key_ssd_cache

ключи в памяти, значения по ним на ssd, есть дополнительные параметры относящиеся к хранению на ssd:

- * block_size

- * file_size

- * read_buffer_size, write_buffer_size

- * path

Как создать словарь

XML-конфигурация

```
<dictionary>  
<name> ИМЯ СЛОВАРЯ </name>  
<structure>  
КЛЮЧ И АТРИБУТЫ  
</structure>  
<source>  
ИСТОЧНИК  
</source>  
<layout>  
СХЕМА ХРАНЕНИЯ  
</layout>  
<lifetime>  
КАК ЧАСТО ОБНОВЛЯТЬ  
</lifetime>  
</dictionary>
```

SQL-синтаксис

```
CREATE DICTIONARY ИМЯ СЛОВАРЯ  
(  
    АТРИБУТЫ  
)  
PRIMARY KEY КЛЮЧ  
SOURCE(ИСТОЧНИК)  
LAYOUT(СХЕМА ХРАНЕНИЯ)  
LIFETIME(КАК ЧАСТО ОБНОВЛЯТЬ)
```


Автообновление

XML:

<lifetime>секунд</lifetime>

или

<lifetime> <min>секунд</min>

<max>секунд</max> </lifetime>

задает период обновления словаря, в случае задания через min/max, период обновления рандомизируется между min и max

SQL: LIFETIME(секунд) или

LIFETIME(***MIN*** секунд ***MAX*** секунд)

Как обращаться к словарю

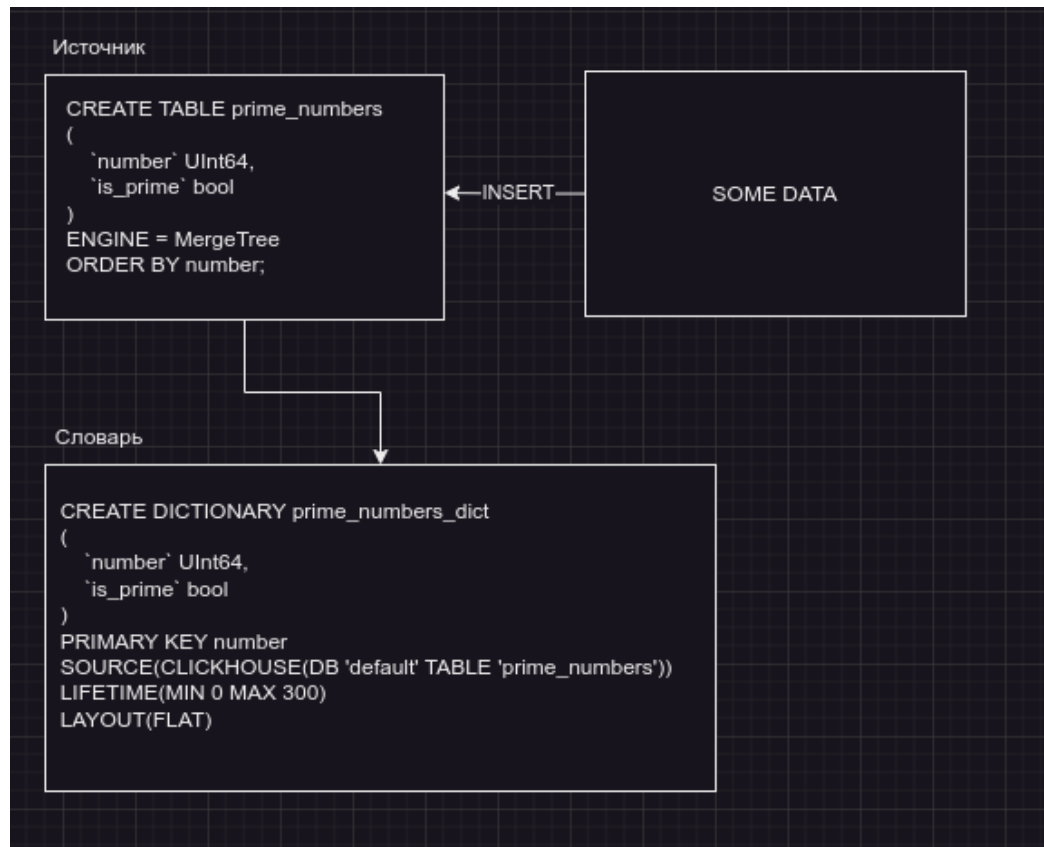
Обращение к словарю выполняется функцией **dictGet()**, принимающей параметры:

dict_name - имя словаря

attr_names - имя атрибута, или Tuple имен атрибутов

id_expr - значение ключа по которому хотим получить атрибуты

Пример



Словарь с простыми числами.

Можно использовать в запросе

```
SELECT
dictGet('prime_numbers_dict','is_prime',
number_column) AS is_prime,
другие колонки
FROM any_table_with_numbers
```

Есть ли альтернативы?

1. **LowCardinality**(string)

2. **transform**(field_id, [index1, index2, index3....], ['value1', 'value2', value3], 'valueElse')

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Табличные функции

Что такое табличная функция

Табличная функция создает временную таблицу на лету, для выполнения над ней запроса, и может использоваться вместо таблицы в секции FROM запроса. Некоторые табличные функции позволяют так же вставку, например функция `file()`.

Популярные табличные функции:

file('путь', 'формат', структура, сжатие)

пример: `SELECT * FROM file( 'path/file.csv', 'CSV', 'column1 UInt32, column2 UInt32' )`

делает таблицу из файла по пути относительно `<user_files>` каталога в конфигурации.

путь - относительный путь до файла

формат - любой из понимаемых ClickHouse (<https://clickhouse.com/docs/en/interfaces/formats>)

структура - список «имя тип» для колонок, разделенный запятыми

сжатие - поддерживаются «gz, br, xz, zst, lz4, bz2», если не указано то никакое

fileCluster(cluster_name, ...) - то же самое что и `file`, но объединит файлы с каждой реплики указанного кластера

merge(['db_name'], 'tables_regexp') - позволяет поработать с несколькими таблицами отобранных по *tables_regexp*, из указанной базы, как с единой таблицей. Базу можно не указывать, тогда будет текущая. Можно указать в место базы так же *regexp*.

url(URL [format] [structure] [headers]) - то же, что и *file()*, но позволяет получить по *http*, можно передавать свои заголовки в формате *'headers('key1'='value1', 'key2'='value2')*

s3(
URL,
[NOSIGN | access_key_id, secret_access_key, [session_token]],
[format] [structure] [,compression_method])
) - как *URL*, только позволяет работать с *s3*

hdfs(URI, format, structure) - то же самое для *HDFS*

merge(['db_name'], 'tables_regexp') - позволяет поработать с несколькими таблицами отобранных по *tables_regexp*, из указанной базы, как с единой таблицей. Базу можно не указывать, тогда будет текущая. Можно указать в место базы так же *regexp*.

url(URL [format] [structure] [headers]) - то же, что и *file()*, но позволяет получить по *http*, можно передавать свои заголовки в формате *'headers('key1'='value1', 'key2'='value2')*

s3(
URL,
[NOSIGN | access_key_id, secret_access_key, [session_token]],
[format] [structure] [compression_method])
) - как *URL*, только позволяет работать с *s3*

hdfs(URI, format, structure) - то же самое для *HDFS*

remote(addresses_expr, [db, table, user [, password], sharding_key]) - позволяет подключиться к другому ClickHouse или нескольким, объединив результаты с таблиц из нескольких ClickHouse
addresses_expr - разделенный запятыми список host:port строк

cluster, clusterAllReplicas - то же самое, но вместо ***addresses_expr*** топология кластера из <remote_servers> конфигурации. Для cluster берется одна реплика каждого шарда, для clusterAllReplicas все реплики.

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Оконные функции

Что такое оконная функция

group	id	value	ROW	RANGE		
1	1	10			Партиция 1	
1	1	20				
1	2	30				
1	5	40				
1	5	50				
2	1	1			Партиция 2	
2	2	2				
2	3	3				
2	3	4				
2	3	5				
2	4	6				
2	5	7				
2	6	8				
2	7	9				
2	8	10				
3	1	100			Партиция 3	
3	2	200				
3	3	300				

Что такое оконная функция

Функция, позволяющая вычислять значения относительно диапазона других строк. Диапазон называется окном.

```
SELECT <window_function_name>([<argument_list>]) OVER {() |  
<window_name> | (<window_spec>)}  
FROM <table_name>  
  
--WINDOW <window_name> AS (<window_spec>);  
  
<window_spec>:  
[PARTITION BY <partition_column> [, ...]]  
[ORDER BY <order_column> [ASC|DESC] [, ...]]  
    [{RANGE | ROWS} {<frame_start> | BETWEEN <frame_start> AND  
<frame_end>}]
```

Описание фрейма

	ROW	RANGE
CURRENT ROW	текущая строка	~
UNBOUNDED PRECEDING	первая строка во фрейме	~
UNBOUNDED FOLLOWING	последняя строка во фрейме	~
<offset> PRECEDING	<offset> строк назад	строки у которых id меньше, чем текущий id на <offset>
<offset> FOLLOWING	<offset> строк вперед	строки у которых id больше, чем текущий id на <offset>

Пример

```
WITH subq AS
(
    SELECT
        number,
        number % 2 AS odd
    FROM system.numbers
    LIMIT 10
)
SELECT
    number,
    odd,
    sum(number) OVER (PARTITION BY odd ORDER BY number ASC) AS cum_sum_odd
FROM subq
ORDER BY number ASC
```

Query id: 9a55424e-92a7-484d-8da9-49326cb30499

number	odd	cum_sum_odd
0	0	0
1	1	1
2	0	2
3	1	4
4	0	6
5	1	9
6	0	12
7	1	16
8	0	20
9	1	25

10 rows in set. Elapsed: 0.006 sec.

subq - подзапрос возвращающий 10 первых чисел, odd - признак четности

на основании признака четности odd строим окна сортированные по number

получаем коммулятивную сумму для четных, коммулятивную сумму для нечетных

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Рефлексия

Цели вебинара

Проверка достижения целей

1. создавать словари из разных источников
 2. познакомиться с оконными функциями
-
-
-

Вопросы для проверки

1. какой layout вы выберете для составного ключа в условиях нехватки оперативной памяти
2. что такое «окно» в понятии оконной функции

Рефлексия



С какими впечатлениями уходите с вебинара?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**